

火车票务管理系统

统一技术规范

版本 1.0

1 项目概述

火车票务管理系统是一个基于 Qt Widgets 和 C++ 开发的桌面应用程序。项目采用分层架构，以分离界面展示逻辑、业务逻辑和数据库访问逻辑。

本文档是团队范围内的技术基线，用于定义统一的软件架构、模块职责、数据库设计、接口约定、集成规则和开发约束。

所有成员都应按照本规范完成各自模块的实现。若需要修改本文档或共享架构，应在实施前与项目负责人沟通并达成一致。

2 技术栈

本项目使用 C++17 作为编程语言，采用 Qt Widgets 作为桌面界面框架，使用 CMake 进行构建，数据库统一选用 SQLite。版本控制工具为 Git + GitHub，日常开发以 develop 为主分支，稳定发布以 main 为目标分支。

3 项目结构

仓库应保持当前的高层目录结构：

```
TrainTicketSystem/  
src/  
  login/  
  train/  
  ticket/  
  statistics/  
  database/  
  ui/  
include/  
database/  
docs/  
tests/
```

当前约定的核心类命名也属于共享结构的一部分：
LoginManager、*TrainManager*、*TicketManager*、*StatisticsManager*、*DatabaseManager*。

4 软件架构

项目采用如下分层架构：

```
Qt UI  
|  
v  
LoginManager  
TrainManager  
TicketManager  
StatisticsManager  
|  
v  
DatabaseManager  
|  
v  
SQLite
```

架构规则

UI 类不得直接执行 SQL，所有数据库访问必须统一通过 *DatabaseManager* 完成。各个 Manager 类通过公共接口向 UI 提供服务，并负责各自模块内部的业务逻辑处理。不同模块之间不应直接操作彼此的内部数据；若需要调整共享结构、重命名核心类或改变分层方式，必须先经过讨论再实施。

5 模块职责

项目负责人

项目负责人主要负责数据库设计与实现、GitHub 仓库管理、系统集成、代码审查以及 Pull Request 审查，同

时承担团队协作过程中的接口协调和整体推进工作。

程序员 A

模块: *login/*

程序员 A 负责登录窗口、身份验证逻辑、管理员相关功能以及权限管理实现。

程序员 B

模块: *train/*

程序员 B 负责站点 CRUD、车次 CRUD、车次查询以及席位库存管理等与列车信息相关的功能。

程序员 C

模块: *ticket/*、*statistics/*

程序员 C 负责订票、退票、改签、订单历史以及统计分析等票务与报表相关功能。

测试人员

测试人员负责功能测试、文档编写、用户手册整理以及 Bug 报告提交。

6 数据库规范

数据库引擎

: SQLite

版本 1 表

版本 1 的正式表包括 *User*、*Station*、*Train* 和 *Order*。

后续版本可扩展表

若后续需求扩大，可进一步引入 *Ticket*、*Payment*、*Seat* 和 *OperationLog* 等扩展表。

版本 1 设计决策

在首个集成版本中，不单独设置 *Ticket* 表。当前约定为：*Order* 表中的一条记录代表一条购票记录。这样可以简化数据库结构，降低多人协作时的集成风险。若后续需求强制要求座位级建模，可在版本 2 中扩展设计。

User

字段	类型	说明
<i>userId</i>	INTEGER	主键，自增
<i>username</i>	TEXT	唯一，非空
<i>password</i>	TEXT	非空
<i>role</i>	INTEGER	0 游客，1 售票员，2 管理员
<i>enabled</i>	INTEGER	默认值为 1

Station

字段	类型	说明
<i>stationId</i>	INTEGER	主键，自增
<i>stationName</i>	TEXT	唯一，非空

Train

字段	类型	说明
<i>trainId</i>	INTEGER	主键
<i>trainNumber</i>	TEXT	唯一，非空
<i>departureStationId</i>	INTEGER	外键，指向 Station
<i>arrivalStationId</i>	INTEGER	外键，指向 Station
<i>departureTime</i>	TEXT	非空
<i>arrivalTime</i>	TEXT	非空
<i>totalSeats</i>	INTEGER	非空
<i>remainingSeats</i>	INTEGER	非空
<i>enabled</i>	INTEGER	默认值为 1

必要约束：

- remainingSeats* $\dot{=}$ 0
- totalSeats* $\dot{=}$ 0
- remainingSeats* $j=$ *totalSeats*

Order

字段	类型	说明
<i>orderId</i>	INTEGER	主键
<i>userId</i>	INTEGER	外键，指向 User
<i>trainId</i>	INTEGER	外键，指向 Train
<i>passengerName</i>	TEXT	非空
<i>purchaseTime</i>	TEXT	非空
<i>status</i>	INTEGER	0 已预订，1 已退票，2 已改签

关系定义

Order.userId 关联 *User.userId*, *Order.trainId* 关联 *Train.trainId*。同时, *Train.departureStationId* 和 *Train.arrivalStationId* 分别关联 *Station.stationId*, 用于表示车次的出发站和到达站。

7 接口约定

当前共享的核心 *Manager* 类如下:

LoginManager、*TrainManager*、*TicketManager*、*StatisticsManager* 以及 *DatabaseManager*。

各 *Manager* 通过公共接口向 *UI* 提供服务。*DatabaseManager* 是唯一允许执行 SQL 的组件。

预期职责

LoginManager 负责登录验证、角色校验和密码修改;
TrainManager 负责车次与站点的增删改查、车次搜索以及库存更新;
TicketManager 负责订票、退票、改签和历史查询;
StatisticsManager 负责报表与统计汇总;
DatabaseManager 则负责连接管理、数据库模式初始化以及共享查询执行。

8 开发规则

每个功能都应在独立的功能分支中开发, 所有合并必须通过 Pull Request 完成, 禁止直接向 *main* 分支推送。日常开发的主要目标分支为 *develop*。同时, Commit 信息必须清晰描述本次实现内容, 便于审查与追踪。

良好示例: *Implement login validation*、*Add train search*、*Fix refund bug*。

避免使用: *update*、*fix*、*123*。

AI 使用规则

如果团队成员使用 AI 生成代码, 则该 AI 的输出也必须遵循本规范, 保持既有结构一致, 不得擅自发明新的整体架构, 也不得绕开既定模块边界。

9 集成规则

默认情况下, 每位程序员只修改自己负责的模块。若需要跨模块修改, 应提前沟通并说明原因。数据库结构变更必须经过项目负责人批准, 公共接口变更也应在实施

前同步说明。无论模块如何扩展, *UI* 代码都必须保持不直接包含 SQL 逻辑。

10 当前项目状态

项目项	状态
架构设计	已确定
目录结构	已确定
模块职责	已确定
数据库设计	版本 1 已冻结
数据库实现	进行中
SQL 初始化	尚未实现
Qt UI	进行中
测试	尚未开始